

Improving Neural Networks Dropout Using An Enhanced Weights Scaling

Aviv Yehezkel
Tel-Aviv, Israel
aviv@yehezkel.net

Abstract—Dropout is a powerful regularization technique that successfully mitigates overfitting across a wide range of neural network architectures and applications, from computer vision to natural language processing and more. Dropout consists of dropping (i.e., zeroing) a random fraction of the layer's nodes and then scaling the weights according to the complement probability of the dropout rate. This paper proposes an enhanced weight scaling and uses a simulation study over two benchmark datasets to demonstrate its improved accuracy over the standard approach.

Index Terms—Deep-Learning; Dropout; Neural Networks; Regularization

I. INTRODUCTION

Neural networks have achieved great success in various applications, from computer vision to natural language processing, from audio to speech processing, and more. One of the significant challenges is mitigating the network's overfitting using regularization techniques [1]. Among those, Dropout, introduced by Hinton and Srivastava et al. [2], [3], is highly successful and a common practice. The main idea is to randomly drop (i.e., zero) a fraction p of the neural network's nodes during training such that only part of the parameters will be updated. Then, in order to compensate for the dropped nodes, the **standard dropout weights scaling** scales the trained weights by a factor of $1-p$. See comprehensive surveys on dropout in [4], [8].

However, while this standard weight scaling works in expectation, it might negatively affect the model's learning and accuracy. To the best of our knowledge, this question was not part of the previous works on dropout, and this is the first attempt to discuss the effect of dropout weights scaling on the model's learning. In this paper, we present a work in progress that proposes **enhanced dropout weights scaling** method, which is correct per iteration instead of in expectation, and provide an initial simulation study to demonstrate that the proposed approach improves the model's accuracy compared to the standard approach.

The main contributions of this work are as follows:

- 1) First, to the best of our knowledge, this is the first attempt to discuss the issue of dropout weights scaling effect on the model's learning.
- 2) Second, we propose a new approach for dropout weights scaling, which is correct per iteration instead of in expectation.

- 3) Third, we present an initial simulation study over two benchmark datasets to demonstrate the accuracy improvement provided by the proposed weight scaling over the standard approach, which has the potential to affect a wide range of applications.

The rest of the paper consists of the following: Section II discusses related work. Section III will present the new approach. Then, Section IV will provide a simulation study comparing the proposed and standard weight scaling approaches. Finally, Section V will conclude the paper.

II. RELATED WORK

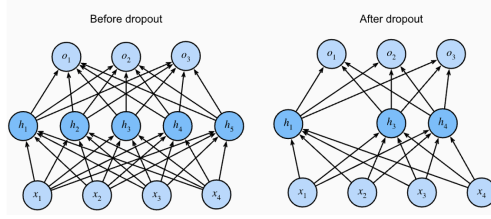
While no previous work directly studied the dropout weights scaling effect, which is the focus of this paper, extensive research on dropout from various angles was conducted, as will be briefly described in this section.

Dropout, introduced by Hinton and Srivastava et al. [2], [3], has become a state-of-the-art regularization method in neural networks. Empirical studies have shown the efficacy of dropout across different domains and tasks and have made it widely adopted in various applications, including computer vision, natural language processing, and reinforcement learning. See [7] for a detailed study of the impact of dropout on model generalization and robustness and comprehensive surveys in [4], [8]. Although the original work described dropping individual neurons in fully connected layers, several extensions of dropout have been proposed over the years to enhance its effectiveness and enable its usage in advanced network architectures such as convolutional layers in convolutional neural networks (CNNs) [5], [9] and recurrent layers in recurrent neural networks (RNNs) [6], [10]–[12].

Another related domain is the study of inconsistency between training and inference of dropout-based models [16]–[18]. However, unlike the proposed approach that improves the standard dropout by enhancing its weight scaling factor and affects only the hidden units of the dropout layer, the works [16]–[18] present new neural network regularization methods that involve both the hidden units and the output of sub-models learned by dropout and do not tackle the dropout weights scaling factor.

III. THE PROPOSED APPROACH

Assume a specific layer's input is denoted by x and output by y , and the layer uses weight matrix W with an activation function $f(\cdot)$.



- For a dropout rate of $p=0.1$, assuming:
- The number of nodes is $|N|=5$
 - The number of dropped nodes is $|D|=2$

Scaling Approach	Scaling Factor
Standard	$1-p = 0.9$
Proposed	$\frac{ N-D }{ N } = 3/5 = 0.6$

Fig. 1. The left side shows an example of dropout taken from [15]. The right side shows the effect of dropout weights scaling. In this example, assuming a dropout rate of $p = 0.1$, the standard scaling factor, which works in expectation, is 50% higher than the enhanced factor proposed in this paper, based on the actual number of nodes that were not dropped.

In order to use dropout for this layer, a Bernoulli random variable is generated per the layer's size such that each neuron gets the value of 0 (i.e., drop) with probability p (dropout rate) and the value of 1 (i.e., keep) with probability $1 - p$. This can be implemented by drawing samples from the uniform distribution and keeping the nodes for which the corresponding sample is larger than p , dropping the rest. Mathematically [4]:

$$y = f(Wx) \circ m, \quad m_i \sim \text{Bernoulli}(1 - p) \quad (1)$$

where m is referred to as the layer dropout mask, as each element m_i being 0 with probability p , meaning its respective neuron is dropped and being 1 otherwise.

Once trained, the standard weight scaling approach [4] scales the trained weights per the expected proportion of the neurons that were not dropped, which is equal to the complement probability of the dropout rate, $1 - p$:

$$y = (1 - p)f(Wx) \quad (2)$$

This scaling considers the average case when fraction p of the nodes will be dropped, and thus, it is correct in expectation. However, in each specific run, the actual number of nodes that will be dropped is a Binomial-distributed random variable. We will denote by N the set of layer nodes and by D the set of dropped nodes in a specific dropout usage (i.e., a subset of N), then, $|D| \sim \text{Binomial}(|N|, p)$. Thus, there might be a gap between the expected value used for the standard weight scaling, $\mathbb{E}(|D|) = p \cdot |N|$, and its actual value. When accumulated over various runs, it might negatively affect the model's learning and accuracy.

We will begin the presentation of the new weight scaling approach using the example in Figure 1. It assumes a dropout rate of $p = 0.1$, resulting in dropping $|D| = 2$ of the $|N| = 5$ hidden layer nodes. The standard weight scaling method will disregard the actual number of dropped nodes and use the expected scaling factor of $1 - p = 0.9$. However, for this specific run, a weight scaling factor of 0.9 will be 50% higher compared to the actual proportion of nodes that were not dropped, which is $\frac{|N-D|}{|N|} = \frac{3}{5} = 0.6$. Intuitively, a scaling factor of 0.9 will reflect a scenario of 0.5 nodes being dropped on average (equal to $p \cdot |N|$). However, as depicted in the Figure, the actual scenario is of $|D| = 2$ nodes being dropped. The new approach will address this gap by considering the actual number of nodes that were not dropped in the specific

run and determining the weight scaling factor accordingly. Thus, for this example, it will result in an accurate weight scaling factor of 0.6.

The new approach is formally presented in Algorithm 1. The main difference is that in each iteration when dropout is used, the weight scaling factor is determined according to the actual proportion of the nodes that were not dropped, which is $\frac{|N-D|}{|N|}$, instead of assuming it to behave according to its expected value $1 - p$. Thus, the new approach is correct per iteration instead of by expectation as the standard approach.

Algorithm 1 Enhanced Dropout Weights Scaling

Input: A neural network layer and a dropout rate p .

Output: y , the layer output.

$D \leftarrow \emptyset$ will be the set of dropped nodes

for i in N **do**

$m_i \sim \text{Uniform}(0, 1)$

if $m_i \leq p$ **then**

$D.append(i)$

end if

end for

Return $y = \frac{|N-D|}{|N|} f(Wx)$

IV. SIMULATION STUDY

This section will present the simulation study results, demonstrating how the proposed approach enhances the model's accuracy when compared to the standard weight scaling method.

To ensure the robustness of the findings, we will compare the proposed and the standard approaches on two widely recognized benchmark datasets. These datasets span two application domains: image classification, a fundamental computer vision task, and binary sentiment detection, a key natural language processing problem. As an image classification dataset, we will use CIFAR-10 [13] consisting of 32x32 images belonging to ten classes. In addition, we will use IMDB [14], a sentiment detection dataset consisting of positive and negative English reviews. We followed best practices to train the models for reasonable evaluations and used the following architectures:

- CIFAR-10: two CNN layers, each followed by ReLU and Max-polling. Then, two fully connected layers were used

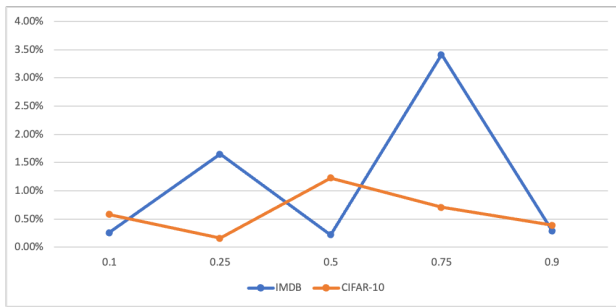


Fig. 2. The average improvement of the proposed approach over the standard weight scaling, per different dropout rates p . Results are averaged over a dozen runs.

with ReLU and a final fully connected layer. The dropout was added after the first fully connected layer.

- IMDB: an embedding layer was followed by an LSTM layer. Then, a sigmoid activation wrapped up a first fully connected layer with ReLU and a second fully connected layer. The dropout was added after the first fully connected layer.

The deep-learning models were based on publicly available implementations¹.

We implemented the proposed method presented in Algorithm 1 and compared it with the standard dropout, i.e., for each test, we trained two models of the same architecture and number of parameters with the only difference relating to the dropout weights scaling method: the first model's weights scaling was done according to the proposed method, and the second model's weights scaling was done based on the standard method. We then ran both models on the same test dataset and reported their accuracy differences. Using the same underlying model's architecture and number of parameters and repeating the simulation multiple times for the two different benchmark datasets, we can compare both methods of dropout weights scaling in an apple-to-apple fashion.

Figure 2 shows the average improvement of the proposed approach over the standard approach: For IMDB, we can see significant accuracy improvements for $p = 0.25$ and $p = 0.75$, reaching a peak of over 3.4% accuracy gain. For the CIFAR-10 dataset, the improvement reached a peak of 1.23% for $p = 0.5$. This initial simulation study shows signs of better model learning through improved accuracy using the proposed weight scaling method over the standard one for both benchmark datasets. It is important to emphasize that even a 1% improvement can be vital in real-world applications, where every accuracy gain can make a difference. Thus, the proposed approach can potentially affect a wide range of applications.

V. CONCLUSION

While dropout has proven to be a successful tool in mitigating neural network overfitting and improving generalization

performance and is commonly used across various applications, this paper considered a new angle by investigating the dropout weights scaling factor. This topic has not been extensively explored to the best of our knowledge. We proposed an enhanced weight scaling based on the actual dropout in each run and used a simulation study of two datasets to demonstrate its improvement in accuracy. Future research will use an extensive empirical study to analyze the accuracy improvement gain over the standard approach, understand when the gain is significant, and derive optimal trade-off relations between the dropout rate, the layer size, the training duration, and the effect on model accuracy.

REFERENCES

- [1] C. F. G. D. Santos and J. P. Papa, "Avoiding Overfitting: A Survey on Regularization Methods for Convolutional Neural Networks," *ACM Computing Surveys*, 2022.
- [2] G. E. Hinton, N. Srivastava, A. Krizhevsky, I. Sutskever, and R. R. Salakhutdinov, "Improving neural networks by preventing co-adaptation of feature detectors," *arXiv:1207.0580*, 2012.
- [3] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A Simple Way to Prevent Neural Networks from Overfitting," *Journal of Machine Learning Research*, 2014.
- [4] A. Labach, H. Salehinejad, and S. Valaee, "Survey of Dropout Methods for Deep Neural Networks," *arXiv:1904.13310*, 2019.
- [5] G. Ghiasi, T. Lin, and Q. V. Le, "DropBlock: A regularization method for convolutional networks," *NIPS*, 2018.
- [6] D. Krueger, T. Maharaj, J. Kram'ar, M. Pezeshki, N. Ballas, N. R. Ke, A. Goyal, Y. Bengio, H. Larochelle, A. C. Courville, and C. J. Pal, "Zoneout: Regularizing RNNs by Randomly Preserving Hidden Activations," *arXiv:1606.01305*, 2016.
- [7] P. Baldi, and P. Sadowski, "Understanding Dropout," *NIPS*, 2013.
- [8] Y. Li, W. Ma, C. Chen, M. Zhang, Y. Liu, et al., "A Survey on Dropout Methods and Experimental Verification in Recommendation," *IEEE Transactions on Knowledge and Data Engineering*, 2022.
- [9] H. Wu and X. Gu, "Towards dropout training for convolutional neural networks," *Neural Networks*, 2015.
- [10] W. Zaremba, I. Sutskever, and O. Vinyals, "Recurrent neural network regularization," *arXiv:1409.2329*, 2014.
- [11] T. Moon, H. Choi, H. Lee, and I. Song, "Rnndrop: A novel dropout for rnns in asr," *IEEE Workshop on ASRU*, 2015.
- [12] S. Semeniuta, A. Severyn, and E. Barth, "Recurrent dropout without memory loss," *COLING*, 2016.
- [13] A. Krizhevsky, "Learning Multiple Layers of Features from Tiny Images," 2009.
- [14] A. L. Maas, R. E. Daly, P. T. Pham, D. Huang, A. Y. Ng, and C. Potts, "Learning Word Vectors for Sentiment Analysis," *Human Language Technologies*, 2011.
- [15] https://d2l.ai/chapter_multilayer-perceptrons/dropout.html
- [16] X. Ma, Y. Gao, Z. Hu, Y. Yu, Y. Deng, and E. Hovy, "Dropout with expectation-linear regularization," *arXiv:1609.08017*, 2016.
- [17] K. Zolna, D. Arpit, D. Suhr, and Y. Bengio, "Fraternal dropout", 2018.
- [18] X. Liang, L. Wu, J. Li, Y. Wang, Q. Meng, T. Qin, W. Chen, M. Zhang, and T. Liu, "R-Drop: Regularized Dropout for Neural Networks," *NIPS*, 2021.

¹ CIFAR-10: https://colab.research.google.com/github/pytorch/tutorials/blob/gh-pages/_downloads/cifar10_tutorial.ipynb
IMDB: https://colab.research.google.com/github/scoutbee/pytorch-nlp-notebooks/blob/master/3_rnn_text_classification.ipynb